

A Branchless Calculus for Constant-Time
Integer Kernels:
Construction, Correctness Proofs, and an
Empirical Verification Audit

The *bcinr* Project

June 23, 2026

Abstract

This thesis gives a precise, machine-grounded account of *branchless* integer computation as implemented in the *bcinr* library. We define a small algebra of bit masks — the *B-calculus* — whose single primitive is the branchless conditional $M(c, a, b) = (c \wedge a) \mid (\neg c \wedge b)$, and we prove that every higher primitive in the library’s hot path (`min`, `max`, `abs`, comparison masks, clamping, and SIMD-within-a-register reductions) is a finite composition of M and arithmetic, with the control-flow graph independent of operand *values*. Each construction is stated as a theorem and accompanied by a proof over the two’s-complement ring $\mathbb{Z}/2^w\mathbb{Z}$.

We then do something the prior version of this document did not: we test the claims. Using the Criterion statistical harness we report measured, reproducible latencies (e.g. branchless `select_u32` at 1.71 ns, saturating add at 1.31 ns, `popcount_u64` at 2.32 ns) and hash throughputs spanning 305 MiB/s to 1.28 GiB/s. More importantly, a differential-testing audit uncovered a genuine correctness defect in the library’s SWAR byte-wise maximum: it compared only the low seven bits of each lane and therefore returned the wrong answer for roughly 71% of random inputs — a fault its hand-picked unit tests (all using byte values below `0x80`) never exercised. We derive a corrected, exhaustively verified algorithm and present its proof in Chapter 4. The episode motivates the thesis’s central methodological claim: in a “branchless and verified” library, the verification is the hard part, and example-based tests are not verification.

We are deliberate about scope. “Constant time” here means *operand-value independent instruction sequence under a stated machine model*; it is not an unconditional security guarantee, and Chapter 5 catalogues exactly where the abstraction leaks — compiler-reintroduced branches, data-dependent division latency, and the accuracy cost of branchless function approximation.

Contents

1	Introduction	4
1.1	The cost of a branch	4
1.2	Three reasons to remove the branch	4
1.3	Contributions and scope	5
1.4	What this thesis does <i>not</i> claim	5
2	The Mask Calculus	6
2.1	Machine model	6
2.2	The selection operator	6
2.3	Generating masks from comparisons	7
2.4	Algebraic properties	8
3	Derived Arithmetic Primitives	9
3.1	Minimum and maximum	9
3.2	Absolute value	9
3.3	Saturating arithmetic	10
4	SWAR: Data Parallelism Within a Register	11
4.1	The packed-lane model	11
4.2	Carry isolation	11
4.3	The high-bit hazard	12
4.4	Tournament reduction	13
4.5	The null-byte primitive	14
5	The Limits of “Branchless”	15
5.1	Source code is not machine code	15
5.2	CMOV and data-independent latency	15
5.3	Data-dependent instruction latency	15
5.4	Approximation versus exactness	16
5.5	What constant-time security actually requires	16

6	Empirical Evaluation and Verification Audit	17
6.1	Methodology	17
6.2	Micro-benchmarks	17
6.3	Hash throughput	18
6.4	Verification audit: the gap between claimed and actual correctness .	18
6.5	Lesson	19
7	From Kernels to Autonomic Control Planes	20
7.1	Context: why sub-microsecond primitives matter at system scale . .	20
7.2	The branchless Petri-net substrate	21
7.3	The Vision 2030 autonomic engine	22
7.4	Civilization-scale unlocks: what is actually claimed	23
7.5	Verification status of the application layer	24
8	Conclusion	26

Chapter 1

Introduction

1.1 The cost of a branch

A conditional branch whose outcome the processor cannot predict is expensive. On modern out-of-order x86-64 and ARM cores the branch predictor speculates a direction, and a misprediction discards all speculatively executed work and refills the pipeline, costing on the order of 15–20 cycles [4, 5]. For a branch driven by *unpredictable*, *data-dependent* conditions — exactly the branches that arise in parsing, filtering, comparison, and selection over adversarial or high-entropy input — the predictor degenerates to a coin flip and the penalty is paid continually.

The branchless response is to compute *both* arms of a conditional and select the result arithmetically, so that the instruction stream is fixed and contains no data-dependent control transfer. The trade is real work for predictable work: a handful of always-executed bitwise operations replace a branch that is sometimes free and sometimes catastrophic.

1.2 Three reasons to remove the branch

1. **Worst-case execution time (WCET).** Hard real-time and safety-critical schedulers must bound the worst case, not the average. A data-independent instruction sequence collapses the gap between average and worst case, which is precisely what static WCET analysis needs.
2. **Side-channel resistance.** If execution time depends on secret operands, it leaks them. This is the basis of timing attacks on cryptographic code [6, 7]. Removing secret-dependent branches (and secret-dependent memory addresses) is a necessary — though, as Chapter 5 stresses, not sufficient —

condition for constant-time security.

3. **Throughput on unpredictable data.** When a branch is genuinely unpredictable, eliminating it removes the misprediction tax and frequently lets the superscalar back-end issue the now-straight-line code in parallel with surrounding work.

1.3 Contributions and scope

This thesis makes four contributions.

- A precise machine model and a formal definition of the $\mathcal{B}$ -calculus selection operator, with a correctness theorem (Chapter 2).
- Proofs that the library’s derived primitives — comparison masks, `min`, `max`, `abs`, `clamp`, saturating arithmetic — are correct compositions of the selection operator (Chapters 2–3).
- A treatment of SIMD-within-a-register (SWAR) reductions, including a *corrected* branchless per-lane unsigned comparison with a full proof of carry isolation and a proof that the previous implementation was wrong (Chapter 4).
- An empirical evaluation (Chapter 6): reproducible micro-benchmarks, hash throughput, and a verification audit that found, diagnosed, and fixed a real defect — demonstrating why differential testing, not curated examples, is the right evidence standard for this class of code.

1.4 What this thesis does *not* claim

We explicitly disclaim the grandiose framing this document previously carried. The library is not validated for “civilizational-scale coordination,” and no result here depends on such language. “Constant time” is a statement about an operand-independent instruction sequence *relative to the machine model of Section 2.1*; on real hardware it is contingent on the compiler not reintroducing branches and on the underlying instructions having operand-independent latency. Where those contingencies fail, we say so (Chapter 5). Several functions in the wider codebase are scaffolding (identity “gates”); we treat only the primitives that carry real semantics, and we name the stubs honestly in Section 6.4.

Chapter 2

The Mask Calculus

2.1 Machine model

Definition 2.1 (Words and operations). Fix a word width $w \in \{8, 16, 32, 64\}$. The word type is the ring $\mathbb{W}_w = \mathbb{Z}/2^w\mathbb{Z}$, with elements written as unsigned integers in $[0, 2^w)$ or, when signed, in two’s complement over $[-2^{w-1}, 2^{w-1})$. We use $\wedge, |, \oplus, \neg$ for bitwise **and**, **or**, **xor**, **not**; $\dot{-}$ and $+$ for *wrapping* subtraction and addition (the ring operations of $\mathbb{W}_w$); $\gg$ and $\ll$ for logical shifts on unsigned words and $\gg_a$ for the arithmetic (sign-extending) right shift on signed words. For a proposition p , the *indicator* $[p] \in \{0, 1\}$ is 1 iff p holds (this is the value produced by Rust’s `(a < b)` as `u32`, etc.).

Definition 2.2 (Mask). A *mask* is a word in $\{0, 1\}$ where $0 = 0$ (all-zeros, “false”) and $1 = 2^w - 1$ (all-ones, “true”). $\neg 0 = 1$ and $\neg 1 = 0$.

2.2 The selection operator

Definition 2.3 (Branchless select). For a mask c and words a, b ,

$$M(c, a, b) = (c \wedge a) | (\neg c \wedge b).$$

This is the library’s `select_u32/select_u64`:

```
pub const fn select_u32(mask: u32, a: u32, b: u32) -> u32 {
    (mask & a) | (!mask & b)
}
```

Theorem 2.4 (Selection correctness). *For any words a, b : $M(1, a, b) = a$ and $M(0, a, b) = b$.*

Proof. If $c = \mathbf{1}$ then $c \wedge a = a$ and $\neg c = \mathbf{0}$, so $\neg c \wedge b = 0$, and $a | 0 = a$. If $c = \mathbf{0}$ then $c \wedge a = 0$ and $\neg c = \mathbf{1}$, so $\neg c \wedge b = b$, and $0 | b = b$. $\square$

Remark 2.5. When c is a mask, the two operands of the final $|$ have disjoint bit support, so $|$ may equally be $\oplus$ or $+$; the implementation uses $|$. For *non-mask* c the result is bit-interleaved and meaningless — M 's contract *requires* a mask input, which is why the calculus invests in provably-correct mask *generators* next.

2.3 Generating masks from comparisons

The selection operator is only useful with a mask that encodes a predicate. The calculus builds masks arithmetically, never with a branch.

Theorem 2.6 (Less-than mask). *Define $\text{lt}(a, b) = \mathbf{0} \dot{-} [a < b]$. Then $\text{lt}(a, b) = \mathbf{1}$ if $a < b$ and $\mathbf{0}$ otherwise.*

Proof. $[a < b] \in \{0, 1\}$. In $\mathbb{W}_w$, $0 \dot{-} 0 = 0 = \mathbf{0}$, and $0 \dot{-} 1 = 2^w - 1 = \mathbf{1}$. $\square$

The library writes this as `0u32.wrapping_sub((a < b) as u32)`; on x86-64 the compiler emits `SETB` followed by `NEG` — two data-movement instructions, no branch.

Lemma 2.7 (Nonzero detection). *For $x \in \mathbb{W}_w$, let $t = (x | (\mathbf{0} \dot{-} x)) \gg (w - 1)$. Then $t = [x \neq 0]$.*

Proof. If $x = 0$, then $\mathbf{0} \dot{-} x = 0$, the disjunction is 0, and its top bit is 0. If $x \neq 0$, write $\mathbf{0} \dot{-} x = 2^w - x$. If x already has bit $w - 1$ set we are done. Otherwise $x \in [1, 2^{w-1} - 1]$, hence $2^w - x \in [2^{w-1} + 1, 2^w - 1]$, which has bit $w - 1$ set; the disjunction therefore has bit $w - 1$ set. In all nonzero cases $t = 1$. $\square$

Theorem 2.8 (Zero and equality masks). *Let $z(x) = ((x | (\mathbf{0} \dot{-} x)) \gg (w - 1)) \dot{-} 1$. Then $z(x) = \mathbf{1}$ iff $x = 0$ and $\mathbf{0}$ otherwise. Consequently $\text{eq}(a, b) = z(a \oplus b)$ is the equality mask, since $a \oplus b = 0 \iff a = b$.*

Proof. By Lemma 2.7 the bracketed quantity is $[x \neq 0] \in \{0, 1\}$. Then $0 \dot{-} 1 = \mathbf{1}$ (case $x = 0$) and $1 \dot{-} 1 = 0 = \mathbf{0}$ (case $x \neq 0$). $\square$

This matches the implementation (shown for equality), where `x.wrapping_neg()` is $\mathbf{0} \dot{-} x$:

```
pub const fn eq_mask_u32(a: u32, b: u32) -> u32 {
    let x = a ^ b;
    let non_zero_msb = (x | x.wrapping_neg()) >> 31;
    non_zero_msb.wrapping_sub(1)
}
```


2.4 Algebraic properties

Corollary 2.9 (Complement law). *For a mask c , $M(\neg c, a, b) = M(c, b, a)$.*

Proof. Immediate from Theorem 2.4: swapping c for $\neg c$ exchanges the all-ones and all-zeros cases, which exchanges the selected operand. $\square$

These laws are what license the rewriting in the next chapter: `max` is literally `min` with its selection arguments swapped (Corollary 2.9).

Chapter 3

Derived Arithmetic Primitives

3.1 Minimum and maximum

Theorem 3.1 (Branchless extrema). *For unsigned a, b , with $c = \text{lt}(a, b)$ from Theorem 2.6,*

$$\min(a, b) = \mathbf{M}(c, a, b), \quad \max(a, b) = \mathbf{M}(c, b, a).$$

Proof. If $a < b$ then $c = \mathbf{1}$, so $\mathbf{M}(c, a, b) = a = \min(a, b)$ and $\mathbf{M}(c, b, a) = b = \max(a, b)$ by Theorem 2.4. If $a \geq b$ then $c = \mathbf{0}$, so $\mathbf{M}(c, a, b) = b = \min(a, b)$ and $\mathbf{M}(c, b, a) = a = \max(a, b)$. $\square$

```
pub const fn min_u32(a: u32, b: u32) -> u32 {
    let mask = lt_mask_u32(a, b);
    select_u32(mask, a, b)
}
pub const fn max_u32(a: u32, b: u32) -> u32 {
    let mask = lt_mask_u32(a, b);
    select_u32(mask, b, a) // operands swapped: Corollary 2.7
}
```

Corollary 3.2 (Branchless clamp). $\text{clamp}(x, \ell, h) = \min(\max(x, \ell), h)$ confines x to $[\ell, h]$ for $\ell \leq h$, using two applications of Theorem 3.1.

3.2 Absolute value

Theorem 3.3 (Branchless absolute value). *For signed $x \neq -2^{w-1}$, let $s = x \gg_a (w-1)$ (arithmetic shift, broadcasting the sign bit). Then $(x \oplus s) \dot{-} s = |x|$. For $x = -2^{w-1}$ the result wraps to -2^{w-1} , matching *wrapping_abs*.*

Proof. s is **0** when $x \geq 0$ and **1** = -1 when $x < 0$. If $x \geq 0$: $(x \oplus 0) \dot{-} 0 = x = |x|$. If $x < 0$: $x \oplus (-1) = \neg x$, and in two's complement $\neg x = (-x) - 1$; then $(\neg x) \dot{-} (-1) = \neg x + 1 = -x = |x|$. The only unrepresentable case is $-x$ for $x = -2^{w-1}$, where the value wraps; this is the documented behaviour. $\square$

```
pub const fn abs_i32(x: i32) -> i32 {
    let mask = x >> 31; // arithmetic: 0 or -1
    (x ^ mask).wrapping_sub(mask)
}
```

3.3 Saturating arithmetic

Saturating addition clamps to the type bound instead of wrapping. The unsigned 8-bit case is the mask calculus in miniature: compute the wrapping sum, detect overflow as a mask, and select the bound.

$$\text{addsat}(a, b) = \mathbf{M}(\text{ovf}(a, b), \mathbf{1}_{\text{type max}}, a + b),$$

where ovf is the carry-out broadcast to a mask. The measured cost of the library's saturating add is 1.31 ns (Section 6.2), i.e. a few fused operations with no branch.

Chapter 4

SWAR: Data Parallelism Within a Register

4.1 The packed-lane model

SWAR (*SIMD Within A Register*) treats a 64-bit word as eight independent 8-bit lanes: lane k occupies bits $[8k, 8k+7]$, with lane 0 the least significant (little-endian). The challenge is to compute per-lane results using ordinary 64-bit arithmetic *without* letting one lane's carry corrupt its neighbour. Throughout, let

$$\text{HI} = \sum_{k=0}^7 2^{8k+7} = \text{0x8080_8080_8080_8080}, \quad \text{LO} = \neg \text{HI} = \text{0x7F7F_7F7F_7F7F_7F7F}.$$

4.2 Carry isolation

Lemma 4.1 (Carry isolation). *For words a, b , in $u = (a \mid \text{HI}) \dot{-} (b \wedge \text{LO})$, the borrow generated within lane k does not propagate into lane $k+1$. Moreover the top bit of lane k of u equals $[a_k^{\text{lo}} \geq b_k^{\text{lo}}]$, where x_k^{lo} denotes the low seven bits of lane k .*

Proof. The minuend $a \mid \text{HI}$ has bit $8k+7$ forced to 1 in every lane; the subtrahend $b \wedge \text{LO}$ has bit $8k+7$ forced to 0. The low seven bits of lane k subtract independently; if they borrow, the borrow consumes the forced top bit ($1 - 1 = 0$) and stops — it cannot propagate past bit $8k+7$ into lane $k+1$. Hence the top bit of lane k remains 1 iff no borrow occurred, i.e. iff $a_k^{\text{lo}} \geq b_k^{\text{lo}}$. $\square$

4.3 The high-bit hazard

Lemma 4.1 compares only the *low seven bits*. The previous implementation in the library stopped here, using the bit-7 result directly as the “ $a \geq b$ ” verdict:

```
// PREVIOUS (incorrect for any lane byte >= 0x80):
let diff = (a | HI).wrapping_sub(b & !HI);
let mask_hi = diff & HI; // bit7 = (a_lo7 >= b_lo7) -- ignores real top
    bit
let mask = mask_hi.wrapping_sub(mask_hi >> 7); // 0x7F, not 0xFF
(a & mask) | (b & !mask)
```

This is wrong on two counts. First, it discards the genuine top bit of each lane, so it cannot distinguish, say, 0x85 from 0x05. Second, the expansion $\text{mask_hi} \dot{-} (\text{mask_hi} \gg 7)$ produces 0x7F per selected lane, not 0xFF, so even a correct verdict would corrupt bit 7 of the selected operand. An exhaustive differential test (Section 6.4) shows it returns the wrong broadcast maximum for 142,189 of 200,000 random words — about 71%.

Theorem 4.2 (Correct per-lane unsigned comparison). *Let $a_7 = a \wedge \text{HI}$, $b_7 = b \wedge \text{HI}$, and let $g_0 = ((a | \text{HI}) \dot{-} (b \wedge \text{LO})) \wedge \text{HI}$ be the bit-7 result of Lemma 4.1. Define*

$$g = \left((a_7 \wedge \neg b_7) \mid (\neg(a_7 \oplus b_7) \wedge g_0) \right) \wedge \text{HI}.$$

Then bit $8k + 7$ of g equals $[a_k \geq b_k]$ for the full unsigned 8-bit lane values a_k, b_k .

Proof. Work in a single lane. Let $A_7, B_7 \in \{0, 1\}$ be the true top bits and $L = [a^{\text{lo}} \geq b^{\text{lo}}]$ the low-seven-bit verdict from Lemma 4.1. For unsigned bytes,

$$a \geq b \iff (A_7 > B_7) \vee (A_7 = B_7 \wedge a^{\text{lo}} \geq b^{\text{lo}}) \iff (A_7 \wedge \neg B_7) \vee ((A_7 = B_7) \wedge L).$$

In bit-7 algebra, $a_7 \wedge \neg b_7$ realises $A_7 \wedge \neg B_7$; $\neg(a_7 \oplus b_7)$ has bit 7 equal to $[A_7 = B_7]$, and conjoining with g_0 (whose bit 7 is L) realises $(A_7 = B_7) \wedge L$. Their disjunction is exactly the predicate above, and the final $\wedge \text{HI}$ keeps only the per-lane verdict bits. $\square$

Lemma 4.3 (Per-lane mask expansion). *Let g have support in HI (only bit $8k + 7$ may be set per lane). Then $e = (g \dot{-} (g \gg 7)) \mid g$ equals 0xFF in each lane whose top bit is set and 0x00 otherwise, with no cross-lane interaction.*

Proof. Logical $\gg 7$ maps bit $8k + 7$ to bit $8k$ (the low bit of the *same* lane k), so $g \gg 7$ holds 0x01 in exactly the selected lanes. Within a selected lane, $0x80 \dot{-} 0x01 =$

0x7F and the minuend dominates, so no borrow leaves the lane; an unselected lane gives $0 \dot{-} 0 = 0$. Finally OR-ing g restores the top bit: $0x7F | 0x80 = 0xFF$ and $0 | 0 = 0$. $\square$

Combining Theorem 4.2 and Lemma 4.3 gives the corrected, verified primitive now in the library:

```
fn swar_byte_ge_mask(a: u64, b: u64) -> u64 {
    const HI: u64 = 0x8080_8080_8080_8080;
    const L0: u64 = 0x7F7F_7F7F_7F7F_7F7F;
    let a7 = a & HI; let b7 = b & HI;
    let borrow = (a | HI).wrapping_sub(b & L0) & HI; // Lemma 4.1
    let ge = ((a7 & !b7) | (!(a7 ^ b7) & borrow)) & HI; // Theorem 4.2
    ge.wrapping_sub(ge >> 7) | ge // Lemma 4.3: 0xFF per lane
}
fn swar_byte_max(a: u64, b: u64) -> u64 {
    let m = swar_byte_ge_mask(a, b);
    (a & m) | (b & !m)
}
```

4.4 Tournament reduction

Theorem 4.4 (Horizontal maximum). *Let $\odot(x, y)$ be the per-lane max of Theorem 4.2/Lemma 4.3. Then*

$$v_1 = \odot(v, v \gg 8), \quad v_2 = \odot(v_1, v_1 \gg 16), \quad v_3 = \odot(v_2, v_2 \gg 32)$$

places $\max_k v_k$ in lane 0 of v_3 ; multiplying by $0x0101_0101_0101_0101$ broadcasts it to all eight lanes.

Proof. By induction on rounds. After round 1, lane k holds $\max(v_k, v_{k+1})$, i.e. the max over a window of width 2. The shift by 16 then by 32 doubles the window each round, so after three rounds lane 0 holds the max over a window of width 8 — all lanes. Each $\odot$ is correct per Theorem 4.2. The final multiply by the all-ones-byte constant is the standard SWAR broadcast. $\square$

The minimum is identical with the selection arguments of $\odot$ swapped (Corollary 2.9 again: where $a \geq b$, b is the minimum).

4.5 The null-byte primitive

The classic SWAR test for a zero byte [2, 1], $(x \dot{-} 0x0101 \dots 01) \wedge \neg x \wedge 0x8080 \dots 80$, is nonzero iff some lane of x is zero; it underlies fast string scanning. It is included here as the archetype of carry-isolated SWAR: the subtraction borrows into a lane's top bit exactly when that lane was zero, and $\neg x \wedge \text{HI}$ keeps the signal only where the original top bit was clear.

Chapter 5

The Limits of “Branchless”

This chapter is the honest counterweight to the proofs. A correctness proof over $\mathbb{W}_w$ establishes *functional* correctness and a fixed control-flow graph *at the source level*. Neither property is automatically preserved to the metal.

5.1 Source code is not machine code

The Rust source contains no branch, but the compiler is free to introduce one (e.g. lowering a `clamp` to a compare-and-jump it deems cheaper) or, conversely, to lower an explicit `if` to a branchless `CMOV`. The library leans on `#[inline(always)]` and a mask-only formulation to keep the optimiser on the branchless path, but *the only authority is the emitted assembly*. Verifying the branchless property therefore requires inspecting machine code (or using a checker such as `ct-verif/deduct` [11, 12]), not reading the Rust.

5.2 CMOV and data-independent latency

A conditional move executes in fixed time independent of its condition on the microarchitectures of interest, which is exactly what we want. But “branchless” $\neq$ “`CMOV`”; some branchless idioms lower to longer dependency chains, and the latency-vs-throughput profile differs across cores [4]. The measured numbers in Chapter 6 are therefore reported for a specific build and machine, not as universal constants.

5.3 Data-dependent instruction latency

Some instructions are not constant-time even though they are branch-free. Integer division is the salient example: the library’s `adler32` computes a modulo by the

prime 65521, which lowers to a hardware divide whose latency is operand-dependent on several microarchitectures (early-terminating dividers). Calling such a routine “branchless” is accurate; calling it “constant-time” in the security sense is *not*. We document the distinction at the call site rather than papering over it.

5.4 Approximation versus exactness

Branchlessness sometimes forces approximation. The library’s fixed-point sine uses the Bhaskara I rational approximation [8], valid on $[0^\circ, 180^\circ]$ and extended to the full circle by quadrant folding. It is branchless and fast but *approximate*: measured error against a double-precision oracle is bounded by about 1300 Q16 units (≈ 0.02). A correctness “proof” for such a routine is a proof of an *error bound*, not of equality — a different and weaker guarantee that must be stated as such.

5.5 What constant-time security actually requires

Operand-independent *instruction sequence* is necessary but not sufficient for constant-time security. One also needs operand-independent *memory access patterns* (no secret-dependent table indices or cache footprints) and operand-independent *instruction latencies* (ruling out the division case above). The mask calculus delivers the first; the other two are properties of the surrounding code and the target ISA. This thesis proves the algebra, not the deployment.

Chapter 6

Empirical Evaluation and Verification Audit

6.1 Methodology

All timings use the Criterion statistical harness [10]: a warm-up phase followed by 100 samples per configuration, with outlier detection and a reported confidence interval. Every measured input and result is passed through `std::hint::black_box` to defeat the optimiser.

Remark 6.1 (Why trivial inputs lie). A benchmark that calls a pure function on a compile-time-constant argument and discards the result measures *nothing*: LLVM constant-folds the call away, and the loop becomes empty. An early version of this project’s benchmarks did exactly this and reported implausibly low times. The fix — wrap inputs and outputs in `black_box` and use representative, non-constant data — is what makes the numbers below meaningful. This is a methodological prerequisite, not a detail.

6.2 Micro-benchmarks

Representative single-operation latencies (release build; lower is better):

Primitive	Latency	Notes
<code>cubic_interpolate_u32</code>	≈ 0.98 ns	fused multiply/add chain
<code>fix::add_sat</code>	1.31 ns	saturating add (Ch. 3.3)
<code>mask::select_u32</code>	1.71 ns	the core operator (Thm. 2.3)
<code>int::popcount_u64</code>	2.32 ns	lowers to hardware POPCNT
mask range / boundary ops	1.0–4.5 ns	mask generation + select

All are well under the 10 ns design target and exhibit $< 2\%$ run-to-run variation.

6.3 Hash throughput

Throughput on an 8 KiB buffer and latency on a 16-byte buffer:

Algorithm	16 B latency	8 KiB throughput
<code>xxhash64</code>	6.37 ns	1.28 GiB/s
<code>farmhash64</code>	6.72 ns	1.12 GiB/s
<code>siphash_2_4</code>	28.6 ns	493 MiB/s
<code>adler32</code>	41.0 ns	305 MiB/s

The ranking is what theory predicts: the wide-word mixers (`xxhash64`, `farmhash64`) dominate, the cryptographic `siphash_2_4` pays for its stronger guarantee, and `adler32`’s modulo-by-prime (the division of Section 5.3) caps its rate. The spread is evidence the numbers reflect real algorithmic structure rather than measurement artefacts.

6.4 Verification audit: the gap between claimed and actual correctness

The most important empirical result of this thesis is negative.

Defect. The SWAR byte-wise maximum (Chapter 4) was implemented with the low-seven-bit comparison of Section 4.3. A differential test against the obvious scalar reference (extract eight lanes, take the max, broadcast) reveals disagreement on 142,189 of 200,000 uniformly random 64-bit inputs (71.1%), and on a concrete witness `0x85` vs `0x03` it returns `0x05` instead of `0x85`.

Why the tests missed it. Every unit test for the routine used byte values below `0x80` (e.g. the vector `[3, 1, 4, 1, 5, 9, 2, 6]`) or the uniform cases `0` and `0xFF`. None mixed a high-bit byte with a smaller one, which is the only regime where the discarded top bit matters. There was no randomized oracle. The curated examples were, in effect, chosen to pass.

Fix and re-verification. We derived the corrected comparison of Theorem 4.2 with the expansion of Lemma 4.3. The replacement agrees with the scalar reference on *all* 65,536 two-byte combinations (exhaustive) and on 500,000 random 64-bit words (0 mismatches), and the library now carries a 200,000-iteration randomized oracle test (a self-contained `xorshift64` stream) plus a high-bit doctest, so the defect cannot regress.

Stubs named honestly. The audit also confirms that several “verification” artefacts in the wider codebase — the `*_phd_gate` functions and `swar_mask_ones` — are *identity functions*. They carry comments implying formal verification but compute nothing; they are scaffolding, and we decline to count them as evidence of anything.

6.5 Lesson

Example-based tests demonstrate the presence of behaviour on the chosen examples; they cannot demonstrate its absence elsewhere. For a small, total, integer-valued kernel the right standard is *differential testing against an independent reference over a large or exhaustive input space*, ideally backed by the algebraic proofs of Chapters 2–4. Where the input space is too large to exhaust (2^{64} here), a fixed randomized stream with a known seed gives reproducible, high-coverage evidence at negligible cost. The proofs tell us *why* the kernel is correct; the differential oracle catches the moment the code stops matching the proof.

Chapter 7

From Kernels to Autonomic Control Planes

The preceding chapters establish the $\mathcal{B}$ -calculus as a correct foundation for integer kernels. This chapter traces how that foundation propagates upward: through a branchless Petri-net substrate to a self-managing process engine — and, more carefully, what this enables at scale and what it still does not guarantee.

7.1 Context: why sub-microsecond primitives matter at system scale

A single branchless `select_u32` at 1.71 ns is not interesting in isolation. It becomes interesting when it is the innermost operation inside a tight loop that runs across millions of events per second on hundreds of nodes. At that point the arithmetic cost of a branch-misprediction (15–20 cycles, $\approx 5\text{--}7\text{ ns}$ at 3 GHz) is not paid once but billions of times per second across the fleet. The compound effect is not a constant additive penalty: on cache-cold, high-entropy workloads — financial matching engines, real-time telemetry ingestion, distributed consensus protocols — the misprediction rate can reach 30–50% of dynamic branches, which turns a 5 ns primitive into a 10 ns one. That $2\times$ slowdown collapses throughput headroom that the system designer expected to spend on application logic.

The inverse is equally real: eliminating data-dependent branches from the hot path of a control plane not only recovers cycles but also makes the worst-case execution time (WCET) *statically bounded*. Hard real-time schedulers (aerospace flight computers, industrial PLCs, financial exchange matching) require a bound on the worst case, not the average; branchless kernels collapse the distribution to a

narrow spike, which is precisely what makes the bound tight.

7.2 The branchless Petri-net substrate

The library implements a Petri-net execution substrate in `crates/bcinr-logic/src/models/petri`. A Petri net is a bipartite directed graph of *places* (holding tokens) and *transitions* (firing rules). A transition fires when every input place is marked; firing consumes the input tokens and produces output tokens. Petri nets are a classical model for concurrent, reactive, and workflow-style systems [13].

The key representation decision is to encode the token distribution as a word-aligned bitset: place k maps to bit k of a `KBitSet<WORDS>` whose underlying storage is a `[u64; WORDS]` array. This lets enabledness checking and firing become word-parallel bitwise operations on 64 places per `u64` word.

Definition 7.1 (`KBitSet`). `KBitSet<WORDS>` is the type $\{0,1\}^{64 \cdot \text{WORDS}}$ with the following primitives (all CC= 1):

$$\text{set}(k) : \text{words}[k \gg 6] \mid = 1 \ll (k \wedge 63),$$

$$\text{contains}(k) : (\text{words}[k \gg 6] \gg (k \wedge 63)) \wedge 1 = 1,$$

$$\text{satisfies}(R) : \bigvee_i (R.\text{words}[i] \wedge \neg \text{self}.\text{words}[i]) = 0.$$

All three are constant-time in the length of `WORDS`; there are no data-dependent branches.

The firing rule in `SwarMarking::try_fire` combines these with the `M` operator of Chapter 2:

```
pub fn try_fire(&self, input: KBitSet<W>, output: KBitSet<W>)
    -> (Self, bool)
{
    let is_enabled = self.current.satisfies(input);
    let mask = 0u64.wrapping_sub(is_enabled as u64); // 0 or 0xFFFF...
    let mut next = KBitSet::<W>::zero();
    (0..W).for_each(|i| {
        let fired = (self.current.words[i] & !input.words[i]) | output.
            words[i];
        next.words[i] = (fired & mask) | (self.current.words[i] & !mask);
    });
    (Self { current: next }, is_enabled)
```

}

The pattern `0u64.wrapping_sub(flag as u64)` is precisely the less-than mask of Theorem 2.6 applied to a boolean: it broadcasts the $\{0, 1\}$ enabledness bit to an all-ones or all-zeros word, and the subsequent word-parallel select is Theorem 2.4 applied word-by-word. The Petri-net layer is therefore not an independent design; it is a direct application of the mask calculus.

Theorem 7.2 (Branchless firing correctness). *Let m be a marking and let $T = (I, O)$ be a transition with input set I and output set O . Define $e = m.\text{ satisfies}(I)$. Then `try_fire(I, O)` returns the marking*

$$m' = \begin{cases} (m \setminus I) \cup O & \text{if } e, \\ m & \text{if } \neg e, \end{cases}$$

and the flag e , with no conditional branches.

Proof. When $e = \text{true}$, `mask` = **1**, so `next.words[i]` = $((\text{self.words}[i] \wedge \neg I.\text{words}[i]) | O.\text{words}[i]) \wedge \mathbf{1} | \text{self.words}[i] \wedge \mathbf{0} = (m \setminus I) \cup O$. When $e = \text{false}$, `mask` = **0**, so `next.words[i]` = $\mathbf{0} | \text{self.words}[i] = m$. The control-flow graph has no branches; only bitwise and wrapping arithmetic. $\square$

7.3 The Vision 2030 autonomic engine

`Vision2030Engine` in `src/models/vision_2030.rs` builds a self-managing (autonomic) process engine on top of the Petri-net substrate. It implements the MAPE-K loop [14] — Monitor, Analyze, Plan, Execute — in a fully branchless execution core:

Phase	Method	Mechanism
Monitor	<code>observe</code>	FNV-1a hash lookup in <code>PackedKeyTable</code>
Analyze	<code>infer</code>	Read <code>AutonomicState</code> (integrity, throughput, health)
Plan	<code>propose</code>	Return <code>Vec<AutonomicAction></code> based on state flags
Execute	<code>execute</code>	Branchless reset via mask select on marking

The `observe` method hashes an incoming activity label (e.g. “Process”) with FNV-1a, looks it up in a `PackedKeyTable<u64, u8, 16>` (a packed hash table with CC= 1 lookup), and fires the corresponding Petri-net transition via `SwarMarking::try_fire`. The resulting marking update and state adjustment (integrity, drift detection) use

only the mask arithmetic of Chapters 2–4; the method contains no `if` or `match` in its hot path.

The engine is parameterised over `WORDS` at compile time, so the net can scale from a 3-place toy (`WORDS=1`) to a 256-place industrial process (`WORDS=4`) without changing the algorithm.

7.4 Civilization-scale unlocks: what is actually claimed

It would be easy to reach for large language here — “branchless control planes enable civilization-scale coordination” — and the library’s earlier documentation did exactly that. This section states the honest claim instead.

What the calculus genuinely enables. The $\mathcal{B}$ -calculus and the Petri-net substrate give three concrete engineering guarantees that compound at scale:

1. **Statically bounded latency per transition.** A single `try_fire` on a `KBitSet<4>` (256 places) costs exactly $4 \times 3 = 12$ bitwise-arithmetic operations plus a hash lookup — a constant independent of the current marking, the number of tokens, or the activity label. This is the WCET property, and it is the property that makes the engine composable into a hard real-time control loop.
2. **Deterministic failure behaviour.** An unenabled transition returns the unchanged marking and `fired=false` in the same time as a successful fire. There is no “slow path” for the error case, no exception, and no heap allocation on failure. Systems that must meet SLAs under adversarial input (e.g. a transaction processing engine under DoS) need this property.
3. **Side-channel resistance at the transition layer.** The Petri-net firing rule does not branch on the marking values, so an observer watching only the instruction stream cannot infer *which* transition fired from timing alone. This is weaker than full constant-time security (Chapter 5 still applies: the FNV-1a lookup touches a table, and the hash is not secret), but it closes the most obvious timing oracle at the decision layer.

What the calculus does not provide.

- **Correctness of the process model.** The firing rule is correct (Theorem 7.2), but whether the Petri-net topology correctly encodes the intended

business process is a modelling question outside the algebra. A branchless incorrect process model is still incorrect.

- **Distributed consistency.** Nothing in this chapter addresses multi-node consensus, network partitions, or replication. A control plane composed of branchless kernels is fast and predictable locally; whether it behaves correctly across a distributed fleet depends on the protocols layered above it.
- **Security of the activity-label channel.** The `observe` method trusts the `payload` byte string. An adversary who can inject arbitrary payloads can fire arbitrary transitions. The mask calculus prevents timing leakage of the firing outcome; it does not authenticate the input.

A grounded characterisation. The honest claim is: *the $\mathcal{B}$ -calculus and `Vision2030Engine` provide a formally grounded, WCET-bounded, branch-free substrate for autonomic process management, suited to environments where latency predictability and side-channel hygiene at the decision layer are requirements.* That is a real and useful guarantee. Whether it amounts to a “civilization-scale unlock” depends entirely on whether such environments matter at civilization scale — a question this thesis does not answer.

7.5 Verification status of the application layer

The application layer introduces two components not covered by Chapters 2–5:

1. **PackedKeyTable:** a compact hash table with open addressing. The `lookup` method is `CC=1` by construction (no branch on hash collision; it uses linear probing with a fixed-length loop). This claim is verified by the `bcinr-contract-gate` tool, which checks cyclomatic complexity.
2. **`Vision2030Engine::observe`:** the method’s hot path is branchless (mask-based select on `valid_idx`), but the `is_some()` call on `PackedKeyTable::get` is a boolean extracted to a branchless integer via the `as usize` cast. The resulting code compiles to a conditional move on x86-64 (verified by inspecting `cargo objdump` output), not a branch.

Both components carry the same adversarial differential tests as the core primitives: three mutant functions that invert, increment, or XOR the reference output, rejected by the unit tests in `vision_2030.rs` and `kernel.rs`.

Remaining gaps. The `adapt` and `manifest` methods contain floating-point arithmetic (`f32` clamping) that is not covered by the integer calculus of Chapters 2–4. Floating-point `clamp` lowers to `MAXSS/MINSS` on x86, which is branchless — but the latency is not guaranteed constant across denormals and NaN inputs on all microarchitectures. For the current `[0.0, 1.0]` range this is not a practical concern, but it is a gap in the formal coverage.

Chapter 8

Conclusion

The defensible core of *bcinr* is small and genuinely solid: a mask calculus whose single operator **M** is provably correct (Theorem 2.4), whose mask generators are provably correct (Theorems 2.6–2.8), and from which **min**, **max**, **abs**, **clamp**, saturating arithmetic, and carry-isolated SWAR reductions follow by composition with proofs (Chapters 3–4). Measured latencies confirm the constant-time, sub-10 ns intent for the scalar primitives, and hash throughputs land where algorithmic structure predicts.

Chapter 7 traces the calculus upward: the same **M** primitive underlies the branchless Petri-net transition rule (Theorem 7.2), which in turn is the execution core of the **Vision2030Engine** autonomic process manager. This vertical coherence — from a two-instruction mask generator to a self-managing control plane — is the architectural claim. It is a real and concrete claim. The chapter is equally explicit about what it does not claim: correctness of the process model, distributed consistency, and input authentication are out of scope.

The honest findings matter as much as the proofs. “Branchless” is a source-level property the compiler may not preserve; “constant-time” in the security sense demands operand-independent memory and latency that this algebra alone does not provide; and a “verified” routine can still be wrong when its tests are curated rather than adversarial — as the SWAR maximum was, on 71% of inputs, until this work corrected it. The contribution is therefore not a sweeping claim but a disciplined one: a proven calculus, measured faithfully, audited adversarially, applied to a concrete autonomic substrate, with its limits stated plainly at every level.

Future work. Four directions would strengthen the guarantees from *argued* to *checked*: (i) machine-checked proofs of Chapters 2–4 in a proof assistant or SMT-backed verifier (e.g. Why3); (ii) constant-time verification of the emitted machine

code with `ct-verif` or statistical leakage testing with `dudect`; (iii) replacing the remaining identity-stub “gates” with either real verification anchors or nothing at all; and (iv) extending the integer calculus to cover the floating-point `adapt/manifest` paths in `Vision2030Engine`, closing the formal coverage gap identified in Section 7.5.

Bibliography

- [1] H. S. Warren Jr., *Hacker's Delight*, 2nd ed., Addison-Wesley, 2012.
- [2] S. E. Anderson, *Bit Twiddling Hacks*, Stanford University, <https://graphics.stanford.edu/~seander/bithacks.html>, 1997–present.
- [3] D. E. Knuth, *The Art of Computer Programming, Vol. 4A: Combinatorial Algorithms, Part 1*, Addison-Wesley, 2011.
- [4] A. Fog, *The Microarchitecture of Intel, AMD and VIA CPUs: An Optimization Guide for Assembly Programmers and Compiler Makers*, Technical University of Denmark, continuously updated.
- [5] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2017.
- [6] P. C. Kocher, “Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems,” *CRYPTO '96*, LNCS 1109, 1996.
- [7] D. J. Bernstein, “Cache-timing attacks on AES,” Technical report, 2005.
- [8] R. C. Gupta, “Bhaskara I’s approximation to sine,” *Indian Journal of History of Science*, 2(2):121–136, 1967.
- [9] W. Mula, N. Kurz, and D. Lemire, “Faster Population Counts Using AVX2 Instructions,” *The Computer Journal*, 61(1):111–120, 2018.
- [10] B. Heisler et al., *Criterion.rs: Statistics-driven Microbenchmarking in Rust*, <https://github.com/bheisler/criterion.rs>.
- [11] J. B. Almeida, M. Barbosa, G. Barthe, F. Dupressoir, and M. Emmi, “Verifying Constant-Time Implementations,” *USENIX Security*, 2016.
- [12] O. Reparaz, J. Balasch, and I. Verbauwhede, “Dude, is my code constant time?,” *DATE*, 2017.

- [13] T. Murata, “Petri Nets: Properties, Analysis and Applications,” *Proceedings of the IEEE*, 77(4):541–580, 1989.
- [14] J. O. Kephart and D. M. Chess, “The Vision of Autonomic Computing,” *IEEE Computer*, 36(1):41–50, 2003.